A Manifest Protocol for Verification Inheritance

Locators, Claim Graphs, and Policy-Qualified Trust for Transformed Scholarly Artifacts

Flyxion
Independent Researcher

August 2026

Abstract

A companion theory paper establishes that whether an earlier verification result may be inherited by a transformed successor of a claim is undecidable in general and decidable only within an explicitly bounded fragment, and that any automatic procedure deciding it must therefore be capable of reporting, honestly, that a transformation falls outside what it can certify. This paper takes that boundary as a design constraint rather than a problem to solve, and specifies a protocol for representing verification inheritance across document transformation: a robust in-band locator pointing to a signed, append-only claim manifest; verification records that distinguish assertion, verification, and transformation attestation as three non-interchangeable signature roles; and a claim-type-indexed policy layer that determines which adjudicators are entitled to certify which classes of correspondence judgment. The protocol invents no new cryptographic primitive, watermarking channel, or provenance ontology; it is compatible with existing statistical watermarking, nanopublications, PROV-O, Verifiable Credentials, and C2PA, and its contribution is the semantics that binds these into a structure obeying the theory paper's boundary. In particular, the protocol never converts the theory paper's OUTSIDE_CERTIFIED_FRAGMENT output into an implicit negative or unsupported positive judgment.

1 Introduction

The companion theory paper shows that no total procedure can decide, for an arbitrary pair of claims, whether a transformation from one to the other preserves the semantic content an earlier verification established, and that this failure is not merely practical but a consequence of undecidability for a sufficiently expressive claim language. It also shows that within a restricted, non-trivial class of transformations, correspondence is decidable by normalization, and gives a three-valued operational output, PRESERVED, NOT_PRESERVED_UNDER_$\mathcal{R}$, and OUTSIDE_CERTIFIED_FRAGMENT, that reports exactly what a bounded certification procedure can and cannot establish.

That result fixes an obligation for any system built on top of it. Such a system must never treat OUTSIDE_CERTIFIED_FRAGMENT as though it were a negative semantic verdict, and it must never allow a robust pointer's mere survival across a transformation to stand in for a positive one. This paper specifies a protocol that satisfies that obligation. It is deliberately narrow in what it claims to contribute. It does not solve the correspondence problem; that problem is either decided by a certified rewrite fragment or it is not decided at all, and the protocol's job is only to represent, faithfully and with appropriate authorization, whichever of those outcomes actually obtains for a given claim and transformation. It borrows its cryptographic and provenance substrate wholesale

from existing work, discussed in Section 8, and adds only the semantics needed to keep generation, authorship, verification, and trust from being conflated.

Section 2 states the design principles the protocol is built to preserve. Section 3 defines the data model: claim records, verification records, and transformation attestations. Section 4 gives the three-layer architecture separating the artifact, its robust commitment, and its manifest. Section 5 specifies the three signature roles and the relations they must not be confused with. Section 6 specifies the policy layer governing which adjudicators may issue which classes of judgment. Section 7 defines dependency propagation over the claim graph and policy-qualified verification queries. Section 8 gives the protocol's core operations and its relationship to existing watermarking, provenance, and trust-management infrastructure. Section 9 states explicitly what the protocol does not solve.

2 Design Principles

Four principles, established across the theory paper and the design discussion preceding it, constrain every decision in this specification and are stated here as the protocol's non-negotiable commitments rather than derived again from scratch.

The locator is not the evidence. A robust in-band commitment establishes only that a transformed artifact points to a particular manifest. It establishes nothing about whether the claims in that manifest still correctly describe the transformed artifact.

Authorship is not generation, and assertion is not verification. A generative system, human or machine, that produces an artifact is not thereby its author in the sense relevant to a scholarly claim, and a party asserting a claim is not thereby a party who has verified it. These are separate acts requiring separate signatures.

Verification is not truth. Every verification record describes a method applied by a specific verifier under specific conditions and yields a result relative to that method; it does not itself certify that the underlying claim is true independent of the method's own assumptions, trusted kernel, or scope.

Failure of certification is not certification of failure. Inherited directly from the theory paper's three-valued output, this principle governs every place in the protocol where a claim's transformed successor cannot be automatically certified: the correct representation is an explicit statement of inapplicability, never a default negative and never a default positive.

3 Data Model

Definition 1 (Claim record)**.** *A claim record is a tuple*

$$R_i = (c_i, A_i, \tau_i, D_i, S_i, V_i),$$

where c_i is a canonical commitment to the claim's content, A_i identifies the asserting party, τ_i is the claim's declared epistemic type (for example, definition, axiom, derived, empirical, cited, conjectural, heuristic, or speculative), D_i is a set of typed dependency edges on which c_i depends (defined immediately below), S_i identifies supporting evidentiary sources, and V_i is the (possibly empty, possibly growing) set of verification records attached to c_i.

Typed dependency edges (schema v2). An earlier version of this protocol represented D_i as a flat set of claim identifiers, $D_i = \{c_j, \ldots\}$. Empirical testing under a preregistered mutation protocol against a formally verified specimen showed this to be insufficient: a flat edge conflates at

least three distinct relations, and at least two mispredictions in that testing traced directly to the conflation rather than to any error in the specimen or the predictions' reasoning. A dependency can be *semantic* (changing c_j can change the proposition c_i asserts), *interface* (constructing or invoking c_i's justification requires c_j's signature or an inhabitant of its type, without c_i's correctness depending on c_j's internal justification), or *proof-term* (c_i's specific certified justification happens to reference c_j, independent of whether c_i's statement mentions c_j at all). These are not exclusive: a single dependency can be both semantic and interface, for instance. The corrected schema is

$$D_i = \{(c_j, \rho_{ij}) : \rho_{ij} \subseteq \{\mathsf{semantic}, \mathsf{interface}, \mathsf{proof_term}\}\},$$

a set of pairs rather than a set of identifiers, with each pair's role set recording every relation that dependency instantiates. Manifests conforming to this schema are versioned `2.0-typed-edges`; a consumer that only understands flat dependency sets may recover one by projecting away ρ_{ij}, at the cost of reintroducing the conflation this revision exists to remove.

Definition 2 (Verification record)**.** *A verification record is a tuple*

$$V = (m, I, O, E, r, \sigma_V),$$

where m identifies the verification method (for example, a specific proof assistant kernel, a computer algebra system, a reproducible numerical pipeline, or a human review protocol), I and O are the method's inputs and outputs (for example, a proof object and its hash), E records the environment and version information needed to reproduce the check, $r \in \{\mathsf{accepted}, \mathsf{rejected}, \mathsf{inconclusive}\}$ is the result, and σ_V is the verifier's signature over $(H(c), m, I, O, E, r)$.

Epistemic type τ_i is a declared classification, not a computed one; it does not itself certify anything and is only as trustworthy as the party who declared it and the policy under which that declaration is accepted, a point made precise in Section 6. Verification records, by contrast, carry their own signed evidence and are the protocol's actual carriers of certification.

Definition 3 (Transformation attestation)**.** *A transformation attestation is a tuple*

$$A_{i,T} = (H(c_i), H(c_i'), T, J, r, e, \sigma_V),$$

where T identifies the transformation applied, J identifies the adjudication regime used to assess whether c_i' corresponds to c_i under T, $r \in \{\mathsf{preserved}, \mathsf{not_preserved}, \mathsf{outside_regime}\}$ is the result, e is supporting evidence for that result (for example, a normal-form computation trace if J is the certified rewrite fragment of the theory paper, or a written justification if J is an informal regime), and σ_V is the attesting party's signature.

The result alphabet of $A_{i,T}$ is deliberately the protocol-level image of the theory paper's $D_{\mathcal{R}}$ output: preserved and not_preserved correspond to PRESERVED and NOT_PRESERVED_UNDER_$\mathcal{R}$ when J is a certified rewrite regime, and generalize to any regime, formal or informal, that supplies a comparably disciplined result; outside_regime is the protocol-level image of OUTSIDE_CERTIFIED_FRAGMENT and must never be silently converted into either of the other two values by any downstream consumer of the manifest.

Following the notation discipline fixed in the theory paper, the protocol distinguishes a computed or proved correspondence relation, written $c_i' \equiv_{\mathcal{L}} c_i$ and populated only when J is a formal regime with a decided or proved semantics, from an attested informal judgment, written $V : c_i' \approx_J c_i$, which is a record of a signed opinion and never silently promoted to the former.

4 Layered Architecture

The protocol separates three layers, each independently replaceable.

$$\text{artifact} \;\leftrightarrow\; \text{robust commitment} \;\leftrightarrow\; \text{signed manifest}.$$

The artifact is whatever document, in whatever format, a reader encounters. The robust commitment is a compact, redundant signal embedded in or attached to the artifact,

$$W(x) = (\text{protocol version}, \text{manifest id}, H(M), \sigma),$$

where M is the manifest and σ is a signature over the tuple. $W(x)$ may be realized by any sufficiently robust carrier, a statistical text watermark, an image or audio watermark, a C2PA-style content-credential field, or ordinary file metadata where robustness against transformation is not required; the protocol is agnostic to the carrier technology, consistent with the theory paper's treatment of watermarking as infrastructure rather than contribution. The manifest M is a signed, append-only document containing the claim graph, that is, the full set of claim records $\{R_i\}$, their dependency edges, their verification records, and any transformation attestations issued against them.

Separating these layers solves a concrete architectural problem raised during the protocol's design: a scholarly document may contain hundreds of claims and a correspondingly large dependency graph, and encoding that entire graph steganographically inside the artifact itself would be both unnecessary and fragile. The embedded commitment need only be robust enough to survive the transformations the artifact is expected to undergo and to locate a manifest that can be fetched, verified, and reasoned over separately.

Example 1 (Manifest growth without artifact rewriting)**.** *A claim c is published with epistemic type* CONJECTURE *and no verification records, $R_c = (c, A, \textsf{CONJECTURE}, \emptyset, S, \emptyset)$. Six months later a formal proof is checked and a verification record v_1 with $r =$ accepted is appended, $V(c) = \{v_1\}$, without altering c_i's content or requiring the original artifact to be republished. If the proof is later found to rely on an error, a further verification record v_2 with $r =$ rejected is appended rather than v_1 being deleted, so the manifest retains*

$$R_c \;\longrightarrow\; R_c + v_1 \;\longrightarrow\; R_c + v_1 + v_2,$$

an append-only epistemic history rather than a single mutable verdict.

5 Signature Semantics

The protocol defines three signature roles that must never be represented by the same key, the same signature, or the same manifest field, because doing so would silently collapse distinctions the design principles in Section 2 require to remain separate.

An *assertion signature*,

$$\sigma_A = \text{Sign}_{sk_A}(H(R_i)),$$

means that party A asserts the claim record R_i as stated. It says nothing about whether A verified c_i, generated it, or merely transcribed it from elsewhere; the record's provenance fields, not the assertion signature, carry that information.

A *verification signature*, embedded in each verification record as σ_V above, means that verifier V performed a specific method against a specific representation of the claim and obtained a specific result. It says nothing about whether V is the same party as A, and nothing about whether V's

verification method itself is trustworthy for the claim type in question; that is a policy question, addressed in Section 6.

A *transformation attestation signature*, embedded in $A_{i,T}$, means that the attesting party asserts a specific correspondence judgment between c_i and c_i' under a specific adjudication regime J. It says nothing about whether that party was authorized to use J for a claim of type τ_i; again, that is a policy question.

Proposition 1 (Non-conflation)**.** *No manifest entry may be interpreted as satisfying more than one of the assertion, verification, and transformation-attestation roles unless it carries the corresponding distinct signature for each role it claims to satisfy.*

This is a protocol invariant rather than a further theorem: it is enforceable by requiring conforming implementations to reject or flag any manifest in which, for example, a single signature is presented as simultaneously establishing authorship and verification, precisely the collapse the design principles in Section 2 rule out.

6 Policy Layer and Trust Delegation

A transformation attestation is meaningless on its own; it must additionally be established that the attesting verifier was entitled to use the adjudication regime it invoked, for a claim of the relevant type. The protocol makes this explicit rather than treating a valid signature as sufficient authorization.

Definition 4 (Claim-type policy)**.** *For each epistemic type τ, a policy is a tuple*

$$P_\tau = (\mathcal{J}_\tau, \mathcal{V}_\tau, Q_\tau),$$

where $\mathcal{J}_\tau$ is the set of adjudication regimes accepted for claims of type τ, $\mathcal{V}_\tau$ is the set or class of verifiers, and their associated trust roots, accepted to issue judgments under those regimes, and Q_τ is a quorum or assurance requirement, for example that a formally-typed claim requires a specific recognized proof-assistant kernel and version, or that an empirical claim requires an accepted reproducibility pipeline together with source-data integrity, or that a claim typed as informal prose accepts a single expert human attestation, multiple independent human attestations, or a specified mixed human-and-machine procedure.

Authorization and correspondence are then two separate predicates that must both hold, following the classical separation between trust delegation and the truth of what is delegated:

$$P_\tau \vdash V \text{ authorized to issue judgments under } J, \qquad \text{distinct from} \qquad V \vdash C_J(c, c').$$

The protocol imports this separation rather than proposing a new trust primitive; it is the same distinction formalized in decentralized trust-management systems, applied here specifically to prevent a correspondence judgment from acquiring authority merely because it is signed by some party.

Verification inheritance is then defined conditionally, requiring both an existing verification, a preserved transformation attestation, and satisfied authorization and quorum conditions:

$$V(c_i) \ \wedge \ A_{i,T} = \textsf{preserved} \ \wedge \ J \in \mathcal{J}_{\tau_i} \ \wedge \ V_{\textsf{att}} \in \mathcal{V}_{\tau_i} \ \wedge \ Q_{\tau_i} \text{ satisfied} \implies V_T(c_i'),$$

where $V_T(c_i')$ denotes that c_i' inherits c_i's verification status under transformation T. Absent any one of these conditions, in particular absent an explicit transformation attestation at all,

$$V(c_i) \not\Rightarrow V(c_i').$$

This is the protocol-level statement of the theory paper's central discipline: persistence of a robust locator across a transformation is never, by itself, sufficient grounds for inheritance.

Different institutions may adopt different policies for the same claim type,

$$P_\tau^{(A)} \neq P_\tau^{(B)},$$

in which case the same transformed document may inherit verification under one institution's policy while failing to do so under another's. The protocol treats this as an intended property rather than a defect: it exposes trust assumptions as explicit, comparable policy objects rather than hiding them behind a single, institution-independent verification badge.

7 Dependency Propagation and Policy-Qualified Queries

The claim graph induces standard reachability closures over the dependency relation $c_i \rightsquigarrow c_j$ recorded in each claim record's D_i field. Because D_i is now typed, these closures are role-parameterized: a query must state which relation it is asking about, rather than treating semantic, interface, and proof-term dependence as interchangeable.

Definition 5 (Support and affected closures, role-parameterized)**.** *For a role set* $\rho \subseteq \{\mathsf{semantic}, \mathsf{interface}, \mathsf{proof_term}\}$,

$$\text{Support}_\rho(c) = \{d : d \rightsquigarrow_\rho c\}, \qquad \text{Affected}_\rho(c) = \{c_j : c \rightsquigarrow_\rho c_j\},$$

where $d \rightsquigarrow_\rho c$ *holds when some edge on the dependency path from* d *to* c *carries a role in* ρ*.* $\text{Support}_{\{\mathsf{semantic}\}}(c)$*, in particular, is the closure a reader most often means by "what does* c*'s truth depend on," and is the closure this protocol's earlier, unversioned examples implicitly assumed.*

A policy-qualified verification query then asks whether every claim a target claim depends on satisfies a given verification policy:

$$\text{Verified}_P(c) \iff \forall d \in \text{Support}_{\{\mathsf{semantic}\}}(c),\ V(d) \models P.$$

The role restriction to semantic is deliberate: whether c's justification is still adequately verified is a question about what c's truth depends on, not about which auxiliary lemmas a particular certified proof happened to invoke, or which theorems a caller's construction needed in scope. A protocol implementation that instead evaluated this query over the unrestricted union of all three roles would over-count: it would treat a proof-term or interface dependency as though it were load-bearing for c's meaning, and could report claims as unverified whose assertions never actually depended on the revoked material. Different readers may pose different queries against the same manifest without the manifest itself needing to encode a single privileged notion of verified. One reader may request $P_1 = \{\text{human-reviewed or stronger}\}$, another $P_2 = \{\text{formally machine-checked}\}$, and the manifest answers both by evaluating the same underlying verification records against different policies, rather than by exposing one scalar confidence figure.

The same closures make revocation and re-verification operational rather than merely recorded. If an intermediate claim c_7's formal verification is later found to have been checked against an incorrect formalization, a revocation attestation is appended to c_7's verification record set, and $\text{Affected}_{\{\mathsf{semantic}\}}(c_7)$ identifies exactly the downstream claims whose inherited verification status must be reevaluated, without requiring any claim outside that closure to be touched, and without disturbing claims connected to c_7 only by an interface or proof-term edge, whose own assertions never depended on c_7's content.

Example 2 (Query results distinguishing failure modes)**.** *Given claims* $c_{15}, c_{18}, c_{22} \in \text{Affected}(c_7)$ *and a policy* P *under which* c_7*'s verification is revoked,* $\text{Verified}_P(c_{15})$*,* $\text{Verified}_P(c_{18})$*, and* $\text{Verified}_P(c_{22})$ *all evaluate to false while claims outside* $\text{Affected}(c_7)$ *are unaffected, yielding a response of the form: under policy* P*, claims* $c_1, \ldots, c_{14}$ *remain supported;* c_{15}, c_{18}, c_{22} *no longer do, because their dependency closures include the revoked* c_7*.*

8 Protocol Operations and Interoperability

The protocol exposes four operations. Publish creates a new manifest M from an initial set of claim records, computes and embeds $W(x)$ in the artifact, and signs M. Attest appends a new verification record or transformation attestation to an existing claim in M, without altering or removing prior entries, preserving the append-only property illustrated in Section 4. Query evaluates Support, Affected, or Verified_P against a manifest for a caller-supplied policy P. Revoke appends a revocation attestation to a specific verification record, which does not delete the record but marks it as superseded for the purposes of subsequent Verified_P evaluations, preserving the historical fact that the original verification was once accepted.

None of these operations requires a new cryptographic primitive. Standard collision-resistant hashing and existentially unforgeable signature schemes suffice for H, σ_A, σ_V, and the attestation signatures throughout. The protocol is likewise compatible with, rather than a replacement for, existing infrastructure for each of its layers. The robust commitment $W(x)$ may be carried by any sufficiently robust statistical watermarking scheme, such as sampling-time text watermarking, without modification to this specification. The claim record format is compatible with the nanopublication model of an atomic, signed scientific assertion together with its provenance and publication metadata, and with the W3C PROV data model for representing the provenance relationships among D_i and S_i more generally. Assertion and verification signatures may be realized as Verifiable Credentials, with the asserting or verifying party as issuer and the claim or verification record as the credential's subject. Where an artifact is an image, video, or general media object rather than a mathematical document, the manifest's locator layer may be realized directly via C2PA content-credential fields rather than a bespoke commitment scheme. This protocol's distinct contribution, relative to all of the above, is the claim-level separation of assertion, verification, and transformation attestation, and the policy layer preventing correspondence judgments from acquiring authority merely because they are signed.

9 What This Protocol Does Not Solve

The protocol does not decide correspondence. Whether $C_J(c_i, c_i')$ holds, for any adjudication regime J, is established outside the protocol, by a certified rewrite fragment when J falls within the theory paper's decidable case, or by whatever formal proof, symbolic check, reproducibility pipeline, or human judgment a claim-type policy accepts otherwise. The protocol's obligation is only to represent that result faithfully, with its adjudicator authorized under the relevant policy, and to refuse to convert an outside_regime result into either a positive or negative claim it did not earn.

The protocol does not solve trust bootstrapping. $\mathcal{V}_\tau$, the set of verifiers or verifier classes accepted for a given claim type, must be established by some external process, whether an institutional accreditation scheme, a web-of-trust structure, or a simple fixed list, and this specification is agnostic to which. It also does not solve the semantic-correspondence adjudication problem for informal claims; J regimes for prose remain attested judgments in the sense fixed by the theory

paper's notation, never computed or proved relations, however many independent attestations a policy's quorum requirement collects.

Finally, the protocol does not itself constitute an empirical claim about how well any of this survives real-world transformation. A companion paper measures locator survival, claim correspondence, and verification inheritance as three independent quantities against an actual mathematical document subjected to a controlled battery of benign and adversarial transformations, using this protocol's data model and the rewrite fragment specified in the theory paper to determine, case by case, which of PRESERVED, NOT_PRESERVED_UNDER_$\mathcal{R}$, and OUTSIDE_CERTIFIED_FRAGMENT actually results.

10 Conclusion

This paper specifies a protocol, not a theorem. Its claim records, verification records, and transformation attestations give the theory paper's undecidability result and decidable fragment a place to live in an actual scholarly artifact, and its policy layer prevents the resulting manifest from smuggling authority into a correspondence judgment merely because that judgment is signed. The protocol adds no new cryptography, no new watermarking channel, and no new provenance ontology; its contribution is the semantics binding existing mechanisms into a structure that respects a boundary it did not itself discover. Whether that structure survives contact with a real document, subjected to real transformations, both benign and adversarial, is the question the next paper in this sequence answers empirically.